

Substitution Method for Polynomial Recurrence

Analyzing $T(n) = T(n-1) + n^2$

A step-by-step expansion, mathematical justification, and complexity derivation using the substitution (iterative-expansion) technique.

Presented By:
M Vishnu Vardhan-192472087
CSA0612- Design and Analysis of Algorithm

Problem Definition

Understanding the recurrence relation before solving it

A recurrence relation **expresses the running time of an algorithm on input size n** in terms of its running time on smaller inputs.

- **Recurrence:** $T(n) = T(n-1) + n^2$
- **Base case:** $T(1) = 1$ (constant time)
- **Meaning:** Cost at level n = cost of one smaller sub-problem + n^2 extra work

This models algorithms that do quadratic (n^2) work at each step of a linear recursive chain — e.g. an algorithm that loops over all n elements at every one of n recursive calls.

GOAL OF THIS ANALYSIS

Find a closed-form (Θ) expression for $T(n)$ using the Substitution Method.

- 1 Expand the recurrence repeatedly
- 2 Detect the pattern after k substitutions
- 3 Stop at the base case
- 4 Sum the resulting series
- 5 State the tight bound $\Theta(f(n))$

The Substitution Method — How It Works

A technique of repeatedly replacing $T(n-1)$ with its own recursive definition

1

Expand once

Write $T(n-1)$ using the same recurrence rule: $T(n-1) = T(n-2) + (n-1)^2$

2

Substitute back

Plug this into the original: $T(n) = T(n-2) + (n-1)^2 + n^2$

3

Repeat k times

Continue substituting to reveal a general expression after k steps

4

Apply base case

Stop when the recursive term hits $T(1)$; replace it with its known value

KEY JUSTIFICATION

Each substitution is a valid algebraic replacement because the recurrence $T(n) = T(n-1) + n^2$ holds for every value of $n \geq 2$. Replacing $T(n-1)$, $T(n-2)$, ... with their own definitions does not change the value of $T(n)$ — it only re-expresses it.

Step-by-Step Expansion

Repeatedly substituting $T(n-1)$ reveals the pattern

Step	Expression after substitution	Justification
$k = 0$	$T(n) = T(n-1) + n^2$	<i>Original recurrence</i>
$k = 1$	$T(n) = T(n-2) + (n-1)^2 + n^2$	<i>Substitute $T(n-1) = T(n-2) + (n-1)^2$</i>
$k = 2$	$T(n) = T(n-3) + (n-2)^2 + (n-1)^2 + n^2$	<i>Substitute $T(n-2) = T(n-3) + (n-2)^2$</i>
$k = 3$	$T(n) = T(n-4) + (n-3)^2 + (n-2)^2 + (n-1)^2 + n^2$	<i>Substitute $T(n-3) = T(n-4) + (n-3)^2$</i>
...	...	<i>Pattern continues</i>
k	$T(n) = T(n-k) + \sum i^2 \text{ for } i = n-k+1 \dots n$	<i>General form after k substitutions</i>

Generalizing the Pattern

Stopping the expansion at the base case $T(1)$

General form (after k substitutions):

$$T(n) = T(n-k) + \sum (i^2) \quad \text{where } i \text{ runs from } (n-k+1) \text{ to } n$$

We stop the recursion exactly when it reaches the base case, i.e. when:

$$n - k = 1$$



$$k = n - 1$$

Substituting $k = n - 1$ into the general form:

$$T(n) = T(1) + \sum i^2 \quad \text{for } i = 2 \dots n$$

Since $T(1) = 1$ (a constant), and $\sum i^2$ for $i = 1 \dots n$ already includes the $i = 1$ term ($=1$), this simplifies to:

$$T(n) = \sum i^2 \quad \text{for } i = 1 \dots n$$

Solving the Summation

Applying the standard sum-of-squares formula

Known identity:

$$\sum i^2 \quad (i = 1 \text{ to } n) = n(n+1)(2n+1) / 6$$

Substituting into $T(n)$:

$$T(n) = n(n+1)(2n+1) / 6$$

Expanding the numerator:

$$n(n+1)(2n+1) = 2n^3 + 3n^2 + n$$

$$T(n) = (2n^3 + 3n^2 + n) / 6$$

Dominant term as n grows large:

$$T(n) \approx n^3 / 3 \quad (\text{lower-order terms vanish})$$

WORKED EXAMPLE ($n = 4$)

$$T(4) = T(3) + 4^2$$

$$T(3) = T(2) + 3^2$$

$$T(2) = T(1) + 2^2$$

$$T(1) = 1 \quad (\text{base case})$$

Back-substitute:

$$T(2) = 1 + 4 = 5$$

$$T(3) = 5 + 9 = 14$$

$$T(4) = 14 + 16 = 30$$

$$\text{Formula check: } 4 \cdot 5 \cdot 9 / 6 = 30 \quad \checkmark$$

Final Complexity Result

Expressing the exact solution and its asymptotic bound

Exact closed form:

$$T(n) = n(n+1)(2n+1) / 6$$

$$\Theta(n^3)$$

TIGHT ASYMPTOTIC BOUND

Upper bound

$$O(n^3)$$

Lower bound

$$\Omega(n^3)$$

Order of growth

Cubic in n

Justifying the Result — Proof by Induction

Every substitution step must be verified, not just assumed

BASE CASE

Claim holds for $n = 1$:

$$T(1) = 1(2)(3)/6 = 1 \quad \checkmark \text{ matches } T(1) = 1$$

The formula and the definition agree exactly at the smallest input.

INDUCTIVE HYPOTHESIS

Assume $T(n-1) = (n-1)n(2n-1) / 6$

holds true for $n - 1$.

INDUCTIVE STEP — show it then holds for n

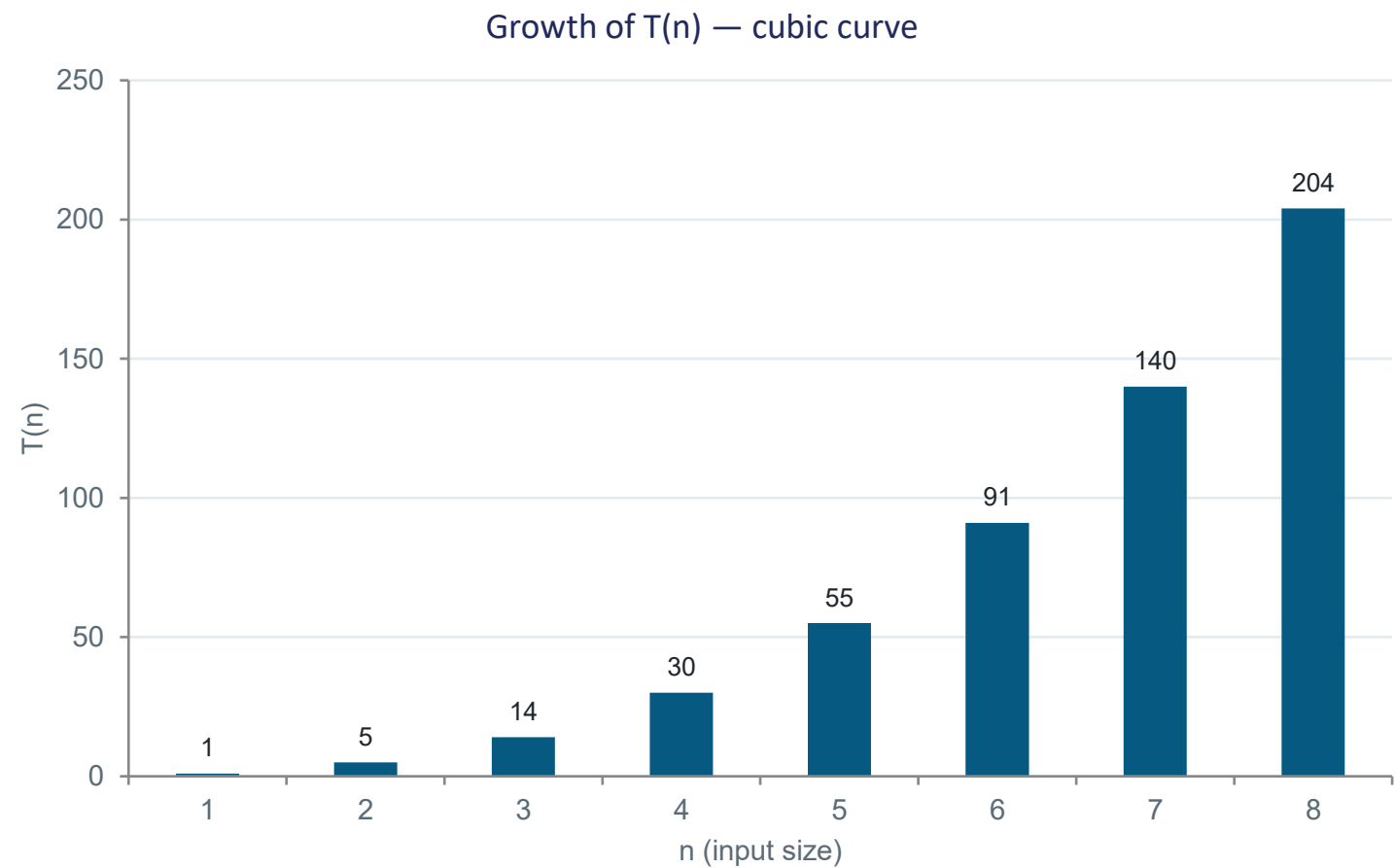
$$\begin{aligned} T(n) &= T(n-1) + n^2 = (n-1)n(2n-1)/6 + n^2 \\ &= [(n-1)n(2n-1) + 6n^2] / 6 = n(n+1)(2n+1) / 6 \end{aligned}$$

This is exactly the closed form evaluated at n — confirming the formula is correct for all $n \geq 1$ by the principle of mathematical induction.

Illustrating the Growth

Numeric values of $T(n)$ versus input size

n	$T(n) = n(n+1)(2n+1)/6$
1	1
2	5
3	14
4	30
5	55
6	91
7	140
8	204



Notice how $T(n)$ accelerates sharply as n grows — the hallmark of cubic, $\Theta(n^3)$, growth.

Conclusion & Efficiency Interpretation

Solved value

$T(n) = \Theta(n^3)$ — a polynomial-time, cubic-order algorithm

Why it grows this way

Linear recursion depth (n levels) $\times$ quadratic work per level (n^2) compounds to cubic total work

Practical efficiency

Acceptable for small-to-moderate n , but scales poorly for large inputs compared to $O(n \log n)$ or $O(n^2)$ alternatives

Where this pattern appears

Naive triple-nested loops, selection sort variants, and certain matrix/graph algorithms with per-step linear scans

The substitution method's strength: it doesn't just guess the answer — it derives and proves it, step by algebraic step.